

NI-KOP – Nasazení Pokročilé Heuristiky

Tomáš Novotný (novott37)

Executive Summary

Experimentální vyhodnocení ukázalo, že implementovaný řešič je vysoce efektivní pro instance do 50 proměnných, kde dosahuje vysoké úspěšnosti v nalezení splňujícího ohodnocení a uspokojivé míry optimality. Pro větší instance (75 proměnných) úspěšnost klesá, ale stále zůstává na použitelné úrovni. Významným pozitivem je robustnost řešiče napříč různými typy instancí a předvídatelná časová náročnost závislá především na velikosti instance.

1 Úvod

Tento úkol se zabývá maximalizačním problémem vážené Booleovy formule, který lze formálně definovat následovně:

Problém 1 (Vážená Booleova formule) Vstup:

- Vektor $\mathbf{X} = (x_1, \dots, x_n)$ binárních proměnných, kde $x_i \in \{0, 1\}$ pro $i = 1, \dots, n$
- Booleova formule F v konjunktivní normální formě obsahující m klauzulí
- Váhová funkce $w : \mathbf{X} \rightarrow \mathbb{R}$ přiřazující každé proměnné x_i váhu $w(x_i)$

Cíl: Nalézt ohodnocení $\mathbf{Y}$ proměnných $\mathbf{X}$ takové, že:

1. $F(\mathbf{Y}) = 1$ (formule je splnitelná)
2. $\sum_{y_i=1} w(x_i)$ je maximální

2 Algoritmus

Na začátek je důležité říct, že účelová funkce, uvedená v Algoritmu 2, je nastavena tak, aby pouze penalizovala nesplněné klauzule. Kandidátním řešením je tedy jakékoliv ohodnocení všech proměnných formule. Základem řešiče je variace simulovaného žíhání k nahlédnutí v Algoritmu 1.

Dále jsou rozebrány jednotlivé části algoritmu včetně všech adaptačních mechanismů.

2.1 Inicializace řešení

Počáteční řešení (stejně tak platí pro počáteční řešení v inicializaci teploty) je získáno nezávislým vygenerováním hodnoty true/false pro každou proměnnou.

2.2 Soused Stavů

Obecně je doporučeno používat pro nalezení souseda mnoho jednoduchých, rychlých a nedrasticky měnících akcí. Dále simulované žíhání vyžaduje jak symetrické operace tak symetrické pravděpodobnosti přechody pro oba směry. Z tohoto důvodu byla zvolena nejjednodušší operace pro změnu kandidátního řešení a to flip ohodnocení jedné proměnné. Rozšířením funkce pro nalezení souseda je zvolení velikosti okolí parametrem U_{size} , který ovlivňuje kolik kandidátních řešení se tímto způsobem vygeneruje a z nich se vybere to s největší účelovou funkcí. Na závěr je důležité zmínit, že každý soused je stejně pravděpodobný.

2.3 Účelová Funkce

Aby byly součástí stavového prostoru všechny kombinace ohodnocení proměnných (a tím byl zaručen lepší pohyb stavovým prostorem) byla zvolena následující účelová funkce uvedená v Algoritmu 2. Je jí vážený součet poměru splněných klauzulí $ratio_{sat}$ a poměru váhy kladně ohodnocených proměnných ku součtu vah všech proměnných $ratio_{weight}$. Váhy složek určují koeficienty f_{weight} a f_{sat} , jejichž adaptaci se věnuje následující Podsekce 2.3.1.

Je dobré zmínit, že poměrem splněných proměnných a poměrem váhy řešení ku maximální možné váze ohodnocení je zaručena adaptace na velikost a parametry instance.

Algorithm 1 Simulated Annealing Solver

Require: Initial solution s_0 , initial temperature T_0 , final temperature T_f

Ensure: Best solution found s_{best}

```
1:  $s_{current} \leftarrow s_0$ 
2:  $s_{best} \leftarrow s_0$ 
3:  $T \leftarrow \text{initializeTemperature}()$ 
4:  $step \leftarrow 0$ 
5:  $noImprovementCount \leftarrow 0$ 
6: while not stopCriteria( $step, T, noImprovementCount$ ) do
7:    $acceptedMoves \leftarrow 0$ 
8:    $equilibriumMoves \leftarrow 0$ 
9:   repeat
10:     $s_{new} \leftarrow \text{getNeighbor}(s_{current})$ 
11:    if acceptTransition( $s_{current}, s_{new}, T$ ) then
12:      updateAdaptiveWeights( $s_{current}, s_{new}, T, \text{true}$ )
13:       $s_{current} \leftarrow s_{new}$ 
14:      if objective( $s_{current}$ ) > objective( $s_{best}$ ) then
15:         $s_{best} \leftarrow s_{current}$ 
16:         $noImprovementCount \leftarrow 0$ 
17:      end if
18:       $acceptedMoves \leftarrow acceptedMoves + 1$ 
19:    else
20:      updateAdaptiveWeights( $s_{current}, s_{new}, T, \text{false}$ )
21:    end if
22:     $step \leftarrow step + 1$ 
23:     $equilibriumMoves \leftarrow equilibriumMoves + 1$ 
24:  until isEquilibrium( $acceptedMoves, equilibriumMoves$ )
25:   $T \leftarrow \text{cool}(step, T)$ 
26:   $noImprovementCount \leftarrow noImprovementCount + 1$ 
27: end while
28: return  $s_{best}$ 
```

Algorithm 2 Calculate Objective Function

Require: Solution s , weight factors (f_{weight}, f_{sat})

Require: Formula F with number of clauses n and maximum weight w_{max}

Ensure: Objective value

```
1:  $satisfied \leftarrow \text{countSatisfiedClauses}(F, s)$ 
2:  $weightSum \leftarrow \text{calculateWeight}(F, s)$ 
3:  $ratio_{sat} \leftarrow satisfied/n$ 
4:  $ratio_{weight} \leftarrow weightSum/w_{max}$ 
5: return  $f_{weight} \cdot ratio_{weight} + f_{sat} \cdot ratio_{sat}$ 
```

2.3.1 Adaptace koeficientů v účelové funkci

Motivací pro následující rozšíření byly instance, které naváděly váhami proměnných k neplatným řešením. Parametrem W je nastavena velikost okna (počet iterací Algoritmu 1, ve kterém je ukládána historie zlepšení, resp. zhoršení, splněných klauzulí u přijatých přechodů na další stav. V každém okně je provedena adaptace vah. Nejdříve je spočten poměr zlepšení počtu splněných klauzulí a celkových kroků v okně. Tento poměr je dále porovnán s parametry $threshold_{low}$ a $threshold_{high}$, kde podle výsledku porovnání dochází ke zvýšení jednoho a snížení druhého koeficientu účelové funkce. Snížování hodnoty koeficientu je navíc zpomalováno v průběhu algoritmu.

2.3.2 Fixní účelová funkce

Adaptace koeficientů účelové funkce má za důsledek, že lepší řešení při nějakých koeficientech účelové funkce může mít nižší hodnotu účelové funkce než horší řešení při jiných koeficientech účelové funkce. Z tohoto důvodu byla představena druhá naivnější účelová funkce, která slouží pouze k nalezení nejlepšího řešení, v Algoritmu 1 na řádce 14, a k zajištění výběru lepšího řešení v následující Podsecei 2.4.

Aby nedošlo k tomu, že v různých iteracích je vní

Algorithm 3 Update Adaptive Weights

Require: Old solution s_{old} , new solution s_{new}
Require: Formula F with number of clauses n and maximum weight w_{max}
Require: Current temperature T , initial temperature T_0
Require: Base adaptive rate α , window size W
Require: Current weight factors: f_{sat} , f_{weight}
Require: Success rate thresholds: $threshold_{low}$, $threshold_{high}$
Ensure: Updated weight factors

- 1: $sat_{old} \leftarrow \text{countSatisfiedClauses}(F, s_{old})$
- 2: $sat_{new} \leftarrow \text{countSatisfiedClauses}(F, s_{new})$
- 3: $\Delta_{sat} \leftarrow (sat_{new} - sat_{old})/n$
- 4: Add Δ_{sat} to history
- 5: **if** solution was accepted and $\Delta_{sat} > 0$ **then**
- 6: Increment $success_{sat}$
- 7: **end if**
- 8: **if** history size $\geq W$ **then**
- 9: $satSuccessRate \leftarrow success_{sat}/W$
- 10: $tempRatio \leftarrow T/T_0$
- 11: $currentAdaptRate \leftarrow \alpha \cdot tempRatio$
- 12: **if** $satSuccessRate < threshold_{low}$ **then**
- 13: $f_{sat} \leftarrow f_{sat} \cdot (1.0 + \alpha)$
- 14: $f_{weight} \leftarrow f_{weight} \cdot (1.0 - \alpha \cdot tempRatio)$
- 15: **else if** $satSuccessRate > threshold_{high}$ **then**
- 16: $f_{sat} \leftarrow f_{sat} \cdot (1.0 - \alpha \cdot tempRatio)$
- 17: $f_{weight} \leftarrow f_{weight} \cdot (1.0 + \alpha)$
- 18: **end if**
- 19: Clamp both factors to $[f_{min}, f_{max}]$ and normalize to sum 100
- 20: Clear history and reset success counters
- 21: **end if**
- 22: **return** (f_{sat}, f_{weight})

2.4 Přechod do dalšího stavu

Přijetí sousedního stavu, v Algoritmu 1 na řádce 11, je důležitým mechanismem pro vymanění se z lokálních optim. Diverzifikace je kontrolována přijetím horšího řešení s následující pravděpodobností:

$$P(\text{accept}) = \begin{cases} 1 & \text{if } \Delta E \leq 0 \\ \exp(-\frac{\Delta E}{T}) & \text{if } \Delta E > 0 \end{cases} \quad (1)$$

Zhoršení ΔE se počítá jednoduše jako rozdíl účelové funkce předchozího stavu a souseda. Intenzifikace je zajištěna zaručeným přijetím lepšího řešení a postupným snižováním pravděpodobnosti přijetí horšího řešení.

2.5 Ekvilibrium

Pro rovnoměrné prohledávání stavového prostoru je přidán mechanismus ekvilibria. Algoritmus prohledává stavový prostor při stejné teplotě a ke chlazení dochází až po přijetí definovaného počtu sousedních řešení nebo při překročení celkového počtu kroků při dané teplotě (tzv. dosažení ekvilibria). Hranice pro počet přijatých řešení je kontrolována parametrem EQ_{total} a hranice pro celkový počet kroků v ekvilibriu je kontrolována parametrem $EQ_{accepted}$.

2.6 Teplotní mechanismus

Konečná teplota je zadána napevno parametrem T_0 . Inspirováno přednáškou [2] bylo nastaveno exponenciální snižování teploty. Aby nemusel být mechanismus snižování teploty závislý na inicializaci teploty nebo mechanismu ekvilibria, používá se následující vzorec pro výpočet teploty v kroku $step$:

$$T_{step} = T_0 \cdot \left(\frac{T_{final}}{T_0} \right)^{\frac{step}{N}} \quad (2)$$

2.6.1 Inicializace Teploty

Na základě přednášky je teplota inicializována výstupem Algoritmu 4. Nejprve se získají hodnoty účelové funkce pro přechody mezi stavy, při kterých dojde ke zhoršení hodnoty účelové funkce. Začne se v náhodném řešení s_0 a postupuje se stavovým prostorem, dokud není dosažen cílový počet zhoršujících přechodů $worse_{size}$ nebo maximální počet iterací $worse_{itermax}$.

Na počáteční hodnotě $T_{initial}$ nezáleží. Iterativně se vylepšuje teplota T dokud nejsme dostatečně blízko žádané pravděpodobnosti úniku z lepšího řešení $escapeProb$. Ta je v každé iteraci spočítaná následujícím vzorcem:

$$P(T) = \frac{\sum_{a \in A} \exp\left(-\frac{E_a^{\max}}{T}\right)}{\sum_{a \in A} \exp\left(-\frac{E_a^{\min}}{T}\right)} \quad (3)$$

Algorithm 4 Temperature Initialization

Require: Initial state s_0 , sample size $worse_{size}$

Ensure: Initial temperature T_0

```
1:  $T \leftarrow T_{initial}$ 
2:  $(worse, prev) \leftarrow \text{findWorseningSteps}(s_0, n)$ 
3: if  $worse$  is not empty then
4:    $escapeProb \leftarrow 1$ 
5:    $iterCount \leftarrow 0$ 
6:   while  $escapeProb \neq targetProb \wedge iterCount < maxIter$  do
7:      $escapeProb \leftarrow P(worse, prev, T)$ 
8:      $factor \leftarrow \ln(escapeProb) / \ln(targetEscapeProb)$ 
9:      $T \leftarrow T \cdot factor$ 
10:     $iterCount \leftarrow iterCount + 1$ 
11:   end while
12: end if
13: return  $T$ 
```

2.7 Ukončovací kritérium

Iterace Algoritmu 1 je ukončena z jednoho z následujících důvodů:

1. pokud teplota T klesne pod hranici T_f ,
2. pokud se teplota snížila $stopNonImprovement$ -krát bez nalezení nového nejlepšího řešení.

2.8 Implementace

Implementace řešiče je umístěna v GitLab repozitáři. Řešič je implementován v jazyce C++. Základní strukturu tvoří několik klíčových tříd, které společně implementují funkcionalitu simulovaného žíhání.

2.8.1 Reprezentace Dat

Pro efektivní reprezentaci problému a jeho řešení byly navrženy následující třídy:

Třída Solution reprezentuje kandidátní řešení problému jako ohodnocení proměnných. Implementace využívá bitový vektor pro paměťově efektivní uložení booleovských hodnot:

```
class Solution {
    std::vector<bool> m_values; // Ohodnocení proměnných
};
```

Třída Clause zapouzdřuje reprezentaci jednotlivých klauzulí formule. Pro efektivní vyhodnocování využívá přímé indexování proměnných, kde záporný index značí negaci proměnné:

```
class Clause {
    std::vector<int> m_literals; // Seznam literálů v klauzuli
};
```

Třída Formula reprezentuje kompletní vstupního MWCNF problém včetně vah proměnných a všech klauzulí:

```
class Formula {
    std::vector<double> m_weights; // Váhy proměnných
    std::vector<Clause> m_clauses; // Seznam klauzulí
};
```

2.8.2 Jádru Řešiče

Třída Solver implementuje hlavní logiku simulovaného žíhání. Obsahuje veškeré mechanismy pro prohledávání stavového prostoru, adaptační mechanismy a řízení teploty:

```
class Solver {
    SolverParameters m_params; // Parametry řešiče
    Formula m_formula; // Instance problému
    PRNG m_gen; // Monte Carlo funkce
    double m_initialTemperature; // Počáteční teplota
public:
    Solution solve(); // Hlavní metoda řešiče
};
```

Třída SolverParameters agreguje veškeré parametry řešiče potřebné pro běh algoritmu

2.8.3 Generování Náhodných Čísel

Třída PRNG zajišťuje deterministické generování náhodných čísel pro Monte Carlo simulaci:

Pro zajištění reprodukovatelnosti výsledků je generátor inicializován pevným seedem při spuštění programu. Veškeré náhodné operace (generování počátečního řešení, výběr souseda, pravděpodobnost přijetí přechodu) využívají výhradně tento generátor.

3 Whitebox Fáze

Pro tuto fázi byla vytvořena redukováná testovací sada instancí pocházející z datasetu [1]. Tato sada obsahuje reprezentativní vzorek instancí ze všech skupin (M, N, P, Q) ve dvou velikostech - 20 proměnných a 50 proměnných, což dohromady tvoří 8 testovacích instancí. Pro eliminaci vlivu náhodnosti algoritmu byl řešič na každé instanci spuštěn dvakrát a byl vybrán lepší z výsledků.

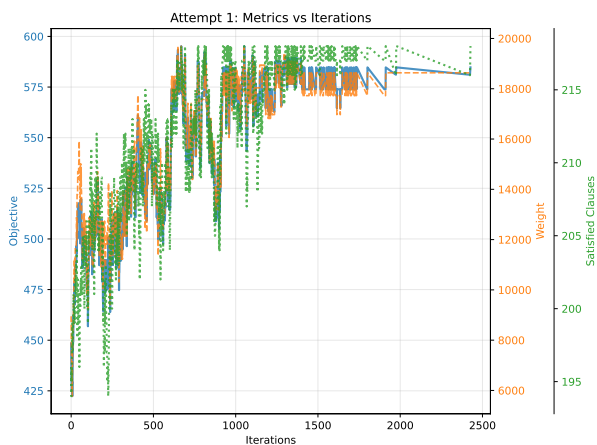
3.1 Systematické Lazení Algoritmu

Proces lazení algoritmu probíhal iterativně s postupným vylepšováním jednotlivých komponent řešiče. Následující text chronologicky popisuje klíčová vylepšení a jejich motivaci. Na Obrázcích 1 jsou k dispozici grafy zobrazující vývoj metrik (účelové funkce, počet splněných klauzulí a váha aktuálního stavu) vzhledem k iteracím v průběhu lazení. Poslední Obrázek 1e zobrazuje finální stav.

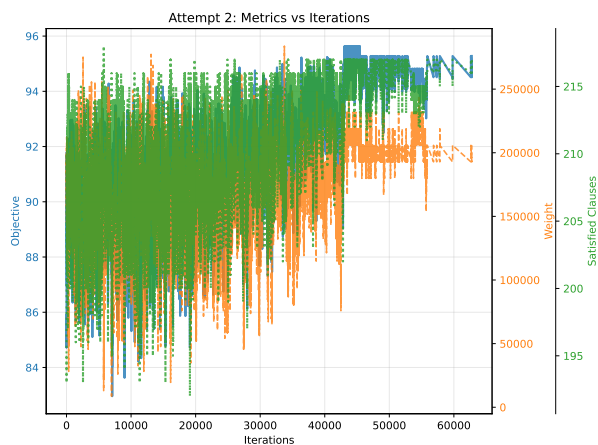
3.1.1 Vývoj Generování Sousedních Stavů

V počáteční implementaci byla pro generování sousedního stavu použita jednoduchá strategie náhodného flipu jedné proměnné. Pro rychlejší konvergenci k ohodnocením splňující formuli byla strategie vylepšena následujícím způsobem:

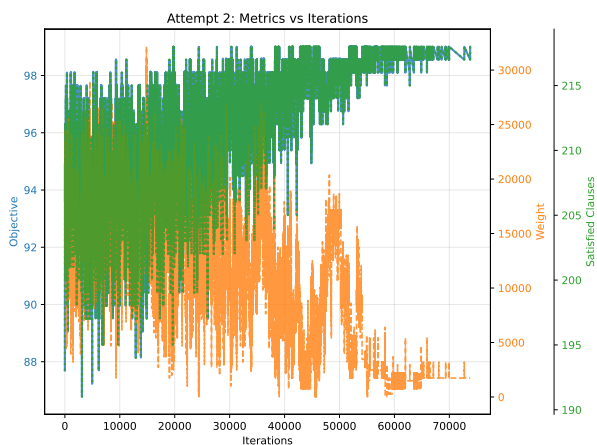
1. Pro stav s nesplněnými klauzulemi se flip provádí pouze na proměnných, které se vyskytují v těchto nesplněných klauzulích. Tím se algoritmus soustředí na problematické části formule.
2. Pro stav se všemi splněnými klauzulemi se zachovává původní náhodný výběr proměnné pro flip. Toto umožňuje další optimalizaci váhové složky účelové funkce.
3. V pozdější fázi bylo implementováno i generování více sousedních stavů s výběrem nejlepšího z nich.



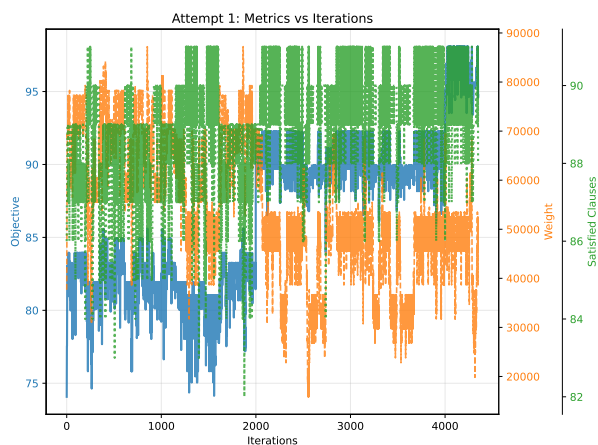
(a)



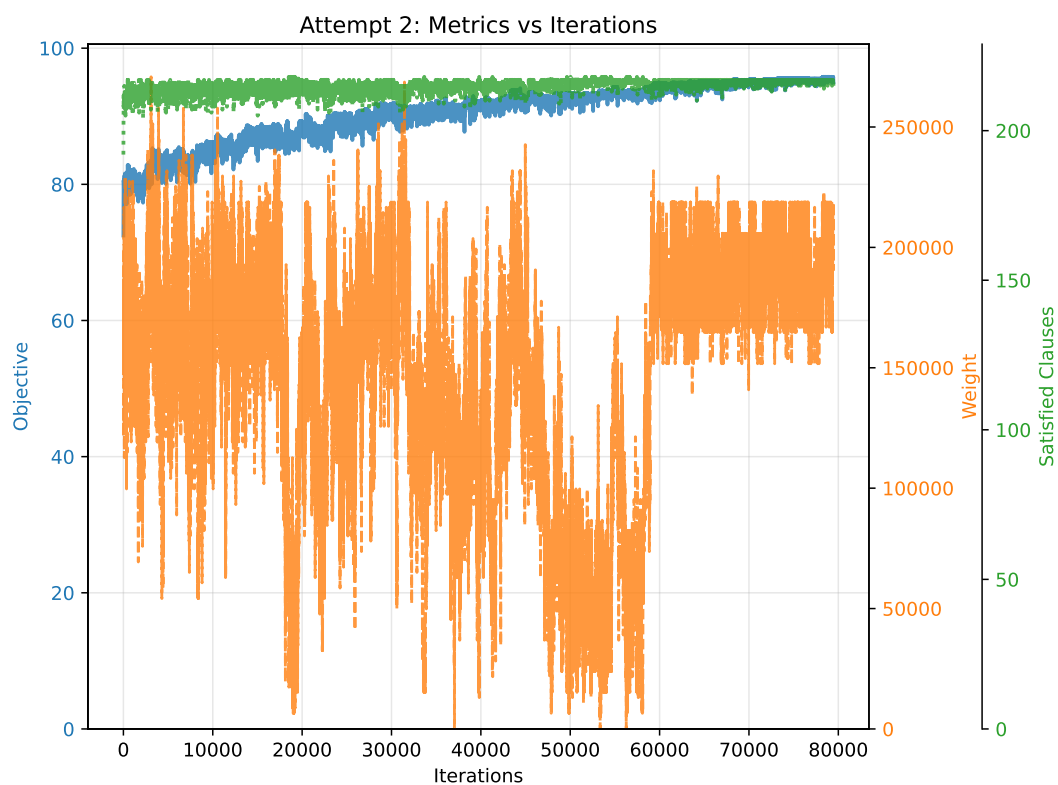
(b)



(c)



(d)



(e)

Obrázek 1: Průběh optimalizace v různých iteracích

3.1.2 Vylepšení Inicializace Teploty

Při implementaci inicializace teploty se objevily problémy s numerickou stabilitou při výpočtu pravděpodobností přijetí přechodů. Tento problém byl vyřešen implementací log-sum-exp triku, který zajišťuje numerickou stabilitu při práci s exponenciálními výrazy:

$$\log \sum_{i=1}^n \exp(x_i) = a + \log \sum_{i=1}^n \exp(x_i - a) \quad (4)$$

kde $a = \max_i x_i$. Tato úprava zajistila spolehlivou inicializaci teploty i pro instance s velkými rozdíly v hodnotách účelové funkce.

3.1.3 Vývoj Přechodové Funkce

Mechanismus přijetí přechodu do nového stavu prošel změnami po přidání adaptace koeficientů v účelové funkci:

1. Původní implementace využívala pouze účelovou funkci pro rozhodování o přijetí přechodu.
2. Později byla přidána fixní účelová funkce pro objektivní porovnání kvality řešení (viz sekce 2.4).

3.1.4 Vývoj Účelové Funkce

Účelová funkce prošla významným vývojem reagujícím na pozorované problémy:

1. Zpočátku byla účelová funkce ve tvaru váh pravdivých proměnných a od toho se odečetla penalizace za nesplnění klauzule. Nejen, že tyto složky nebyly ve stejném měřítku, ale navíc se hodnoty mohly dostat do záporných čísel, což komplikovalo kromě řízení běhu i vizualizace. Budoucí verze účelové funkce byly proto vždy nastaveny jako nezáporné.
2. Následující verze obsahovala vážený součet dvou složek podobně jako v Algoritmu 2 a k tomu se ještě přičetl bonus za splněnou formuli, což nedávalo vůbec dobré výsledky, protože byla moc velká propast mezi splňujícími a nesplňujícími ohodnoceními.
3. Počáteční verze s fixními koeficienty se ukázala jako problematická pro instance se zavádějícími váhami proměnných.
4. Implementace adaptivních koeficientů přinesla významné zlepšení, ale vyžadovala několik iterací optimalizace mechanismu `adjustWeights`. Po několika zesložtích se ukázala původní (aktuálně nasazená) verze jako neefektivnější.
5. Zavedení druhé účelové funkce pro objektivní porovnání řešení (viz výše).

3.1.5 Napravení Řízení Běhu Algoritmu

Původně se čítač kroků (`step`) zvyšoval pouze po dosažení ekvilibria. Toto bylo změněno na inkrementaci po každém prohledání sousedních stavů, což vedlo k plynulejšímu průběhu ochlazování.

3.2 Lazení Parametrů

Na základě zkušeností s řešičem během systematického lazení a nalezených souvislostí mezi parametry se pro prohledání prostoru parametrů zvolily rozsahy uvedené v Tabulce 1. Diskretizace byla zvolena tak, aby výpočet trval řádově hodiny. Všechny kombinace parametrů byly vyzkoušeny na výše zmíněných sadách. Opět byl řešič puštěn nezávisle dvakrát na každou instanci s různými počátečními řešeními a byl vybrán lepší výsledek.

Cílem tohoto experimentu je naléznout kombinaci hodnot parametrů, která řeší nejvíce instancí optimálně a u těch, které neřeší optimálně, je nalezené řešení relativně nejbližší optimálnímu. Tabulka 2 zobrazuje všechny kombinace, které řeší 7 z 8 instancí optimálně. Ve sloupečku *Difference* je uveden rozdíl nalezeného řešení vůči optimu uvedenému ve sloupci *Ref. Weight*. Ve zbylých sloupcích jsou uvedené hodnoty parametrů dané konfigurace.

Mezi parametry nepřevažuje jasný kandidát na nejlepší kombinaci, a proto byla zvolena kombinace uvedená na prvním řádku. S těmito hodnotami pracuje následující Sekce 4.

Tabulka 1: Faktorový Návrh pro Parametry Algoritmu

Fixní parametry		Prohledávané parametry	
Počáteční teplota	1×10^5	Počáteční pravd. úniku	0,5 až 0,9
Přesnost pravděpodobnosti úniku	0,01	Velikost okolí	1 až 5
Velikost vzorku zhoršujících se kroků	1×10^3	Faktor splněných klauzulí	50 až 90
Max. iterace inic. teploty	1×10^4	Míra adaptace	0,01 až 0,1
Konečná teplota	0,1	Dolní hranice úspěšnosti SAT	0,2 až 0,5
Max. počet kroků v ekvilibriu	$1 \times n$	Horní hranice úspěšnosti SAT	0,7 až 0,95
Max. počet přijetí v ekvilibriu	$0.1 \times n$	Délka okna	100 až $500 \times n$
Max. iterace žhání	1×10^5		
Maximální počet kroků teploty bez zlepšení	$100 \times n$		
Plán chlazení	geometrický		

Tabulka 2: Kandidáti mezi konfiguracemi proměnnými a jejich výsledky v experimentu

File Type	Ref. Weight	Difference	Initial Escape Prob.	Satisfied Clause Factor	Adaptive Rate	Success Rate (Low-High)	Window Length
50M	20150	303	0.9	90.0	0.1	0.2-0.95	100.0
50Q	4190	736	0.5	50.0	0.01	0.2-0.95	500.0
50Q	4190	736	0.5	50.0	0.01	0.5-0.7	500.0
50Q	4190	736	0.5	50.0	0.01	0.5-0.7	100.0
50Q	4190	736	0.5	50.0	0.1	0.5-0.7	500.0
50Q	4190	736	0.5	90.0	0.01	0.2-0.7	500.0
50Q	4190	736	0.5	90.0	0.01	0.5-0.95	500.0
50Q	4190	736	0.9	90.0	0.1	0.5-0.7	500.0
50N	264936	3984	0.9	50.0	0.01	0.2-0.95	500.0
50N	264936	3984	0.5	50.0	0.1	0.2-0.7	100.0
50N	264936	5976	0.5	90.0	0.1	0.5-0.7	100.0
50R	36854	6500	0.5	50.0	0.1	0.2-0.95	100.0
50R	36854	6500	0.5	90.0	0.1	0.2-0.95	500.0
50R	36854	6500	0.5	90.0	0.1	0.2-0.95	100.0
50R	36854	6500	0.5	90.0	0.1	0.5-0.95	500.0
50R	36854	6500	0.5	50.0	0.1	0.2-0.95	500.0
50R	36854	6500	0.9	50.0	0.1	0.5-0.7	500.0

4 Blackbox Fáze

4.1 Metody a Materiály

Použitý algoritmus je detailně popsán v Sekci 2. Jednou z klíčových vlastností implementovaného řešení je jeho schopnost adaptace na velikost a charakter řešené instance. Adaptační mechanismy lze rozdělit do následujících kategorií:

1. **Adaptace účelové funkce** - dynamické váhy složek účelové funkce v závislosti na úspěšnosti hledání splňujícího ohodnocení.
2. **Automatická inicializace teploty** - nastavení počáteční teploty na základě charakteristiky instance pomocí statistického vzorkování.
3. **Inteligentní výběr souseda** - prioritizace proměnných vedoucích ke splnění nesplněných klauzulí s ohledem na strukturu instance.

Řešič dále pracuje s velikostí instance pro škálování některých parametrů:

1. Maximální počet kroků v ekvilibriu ($n \cdot \text{equilibriumTotalFactor}$)
2. Maximální počet přijatých řešení v ekvilibriu ($n \cdot \text{equilibriumAcceptedFactor}$)
3. Maximální počet kroků bez zlepšení ($n \cdot \text{stopNonImprovementFactor}$)
4. Délka adaptačního okna ($n \cdot \text{windowLenFactor}$)

Finální hodnoty parametrů řešiče jsou shrnuty v Tabulce 3. Oproti whitebox fázi byla provedena pouze jedna změna - mírné snížení parametru *satisfiedClauseFactor* na základě empirických pozorování.

Tabulka 3: Hodnoty Parametrů Řešiče pro Blackbox Fázi

Parametr	Hodnota	Popis
<i>Parametry teplotního režimu</i>		
initialTemp	100000	Počáteční teplota pro inicializaci
finalTemp	0,1	Koncová teplota pro ukončení žhání
schedule	geometrický	Schéma snižování teploty
<i>Parametry inicializace teploty</i>		
escapeProbAccuracy	0,01	Požadovaná přesnost pravděpodobnosti úniku
worseningSampleSize	1000	Velikost vzorku pro zhoršující přechody
maxIterationsTempInit	10000	Maximum iterací při inicializaci teploty
initialEscapeProb	0,9	Cílová počáteční pravděpodobnost úniku
<i>Parametry ekvilibria</i>		
equilibriumTotalFactor	1,0	Faktor pro celkový počet kroků v ekvilibriu
equilibriumAcceptedFactor	0,1	Faktor pro počet přijatých kroků v ekvilibriu
<i>Parametry prohledávání</i>		
maxIterationsAnnealing	100000	Maximální počet iterací žhání
stopNonImprovementFactor	100	Faktor pro zastavení při stagnaci
neighborhoodSize	1	Počet generovaných sousedů v jedné iteraci
<i>Parametry adaptivní účelové funkce</i>		
satisfiedClauseFactor	80	Váhový faktor pro splnění klauzule
adaptiveRate	0,1	Rychlost adaptace vah
satSuccessRateLow	0,2	Dolní práh úspěšnosti splnění klauzulí
satSuccessRateHigh	0,95	Horní práh úspěšnosti splnění klauzulí
windowLenFactor	100	Faktor pro délku adaptačního okna

4.1.1 Testovací Dataset

Pro experimentální vyhodnocení bylo z datasetu [1] náhodně vybráno 743 instancí. Dataset pokrývá instance následujících velikostí:

- 20 proměnných
- 36 proměnných
- 50 proměnných
- 75 proměnných

Pro každou velikost jsou zastoupeny všechny typy instancí (M, N, P, Q). Pro eliminaci vlivu stochastické povahy algoritmu byl řešič na každé instanci spuštěn vícekrát a bylo vybráno nejlepší nalezené řešení.

4.2 Výsledky a Diskuze

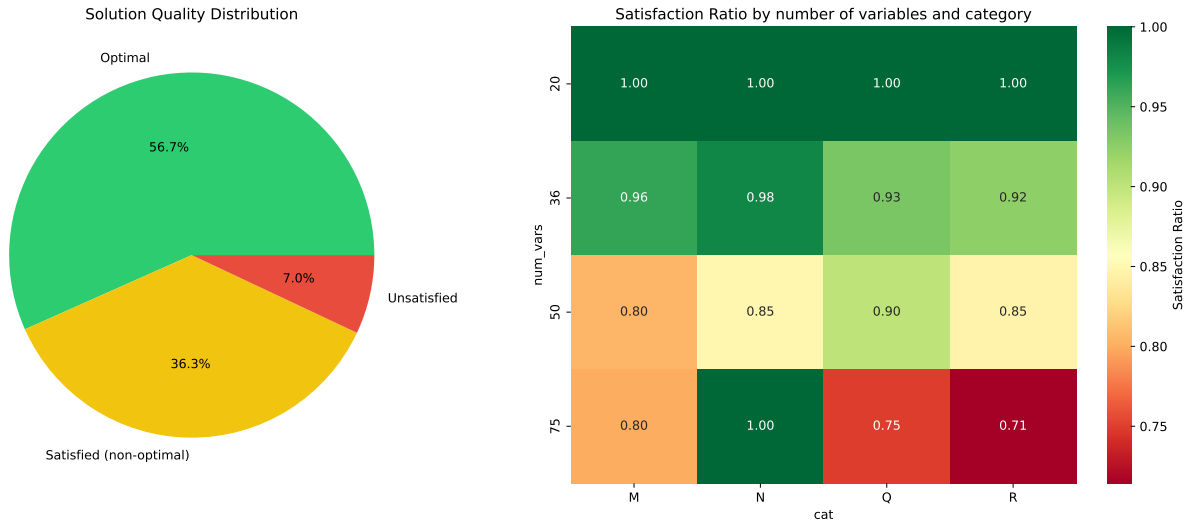
Experimentální vyhodnocení se zaměřilo na tři klíčové aspekty výkonnosti řešiče:

1. Schopnost nalézt splňující ohodnocení
2. Kvalitu nalezených řešení vzhledem k optimu

4.2.1 Nalezení Splňujícího Ohodnocení

Jak je patrné z Obrázku 2, úspěšnost řešiče v nalezení splňujícího ohodnocení je výrazně závislá na velikosti instance:

- Instance s 20 a 36 proměnnými: téměř 100% úspěšnost
- Instance s 50 proměnnými: více než 80% úspěšnost
- Instance se 75 proměnnými: přibližně 70% úspěšnost



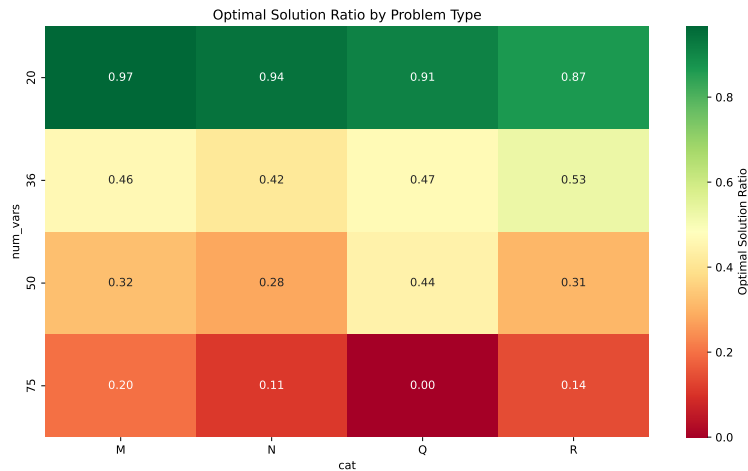
Obrázek 2: Vizualizace Výsledků v Úspěšnosti Splnění Formulí

4.2.2 Optimalita Řešení

Obrázek 3 ukazuje schopnost řešiče nacházet optimální řešení:

- Instance s 20 proměnnými: vysoká úspěšnost v nalezení optima
- Instance s 36-50 proměnnými: 30-50% optimálních řešení
- Instance se 75 proměnnými: minimální úspěšnost v nalezení optima

Obrázek 4 odhaluje distribuci relativních rozdílů optim a váh nalezených řešení. Ukazuje se, že relativní chyba u většiny nalezených řešení se koncentruje do 20%. Významným pozorováním je konzistentní výkonnost řešiče napříč různými typy instancí (M, N, P, Q) stejné velikosti, což potvrzuje robustnost implementovaných adaptačních mechanismů.



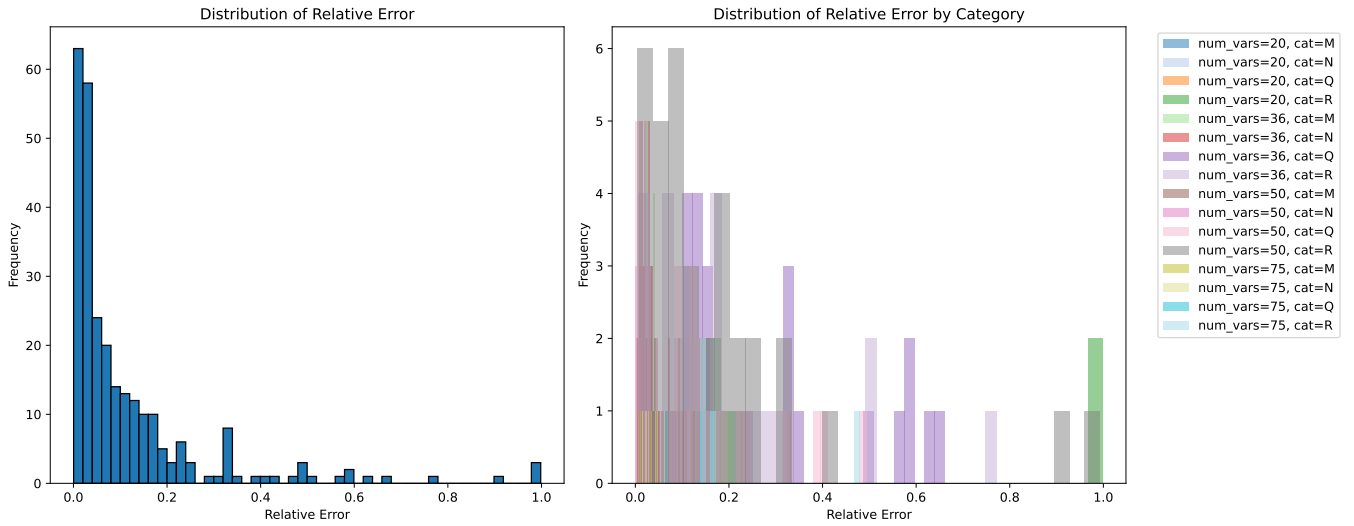
Obrázek 3: Úspěšnost v Nalezení Optimálního Řešení

4.2.3 Časová Náročnost

Experimentálně byla zjištěna následující závislost výpočetního času na velikosti instance:

- 20 proměnných: ~1 sekunda
- 50 proměnných: ~10 sekund
- 75 proměnných: ~50 sekund

Výpočetní čas je primárně ovlivněn počtem proměnných, přičemž vliv strukturální složitosti instance je druhořadý. Toto chování odpovídá očekávání vzhledem k implementovaným adaptačním mechanismům.



Obrázek 4: Distribuce Relativní Chyby u Nalezených Suboptimálních Řešení

5 Závěr

Experimentální vyhodnocení ukázalo, že implementovaný řešič je vysoce efektivní pro instance do 50 proměnných, kde dosahuje vysoké úspěšnosti v nalezení splňujícího ohodnocení a uspokojivé míry optimality. Pro větší instance (75 proměnných) úspěšnost klesá, ale stále zůstává na použitelné úrovni. Významným pozitivem je robustnost řešiče napříč různými typy instancí a předvídatelná časová náročnost závislá především na velikosti instance.

Reference

- [1] Czech Technical University in Prague. Data a programy, 2025. Accessed: 2025-01-15.
- [2] Jan Schmidt. Předmět ni-kop, 2025.